OptiLine

Raceline Optimisation Toolkit

Package Manual — Version 0.1.0

Amir Ali Farzin

amirali.farzin@anu.edu.au

Abstract

OptiLine is a Python package for computing optimal racing lines on closed tracks. It provides routines for minimum-curvature and minimum lap-time trajectory optimisation, shortest-path planning, cubic and clothoid spline geometry, and full kinematic profile generation (velocity, acceleration, and lap time). The package combines quadratic programming, evolutionary strategies (CMA-ES), zero-order Gaussian random methods (ZORM), and direct collocation via CasADi/IPOPT into a unified, extensible interface aimed at autonomous racing research.

Python $\geq$ 3.10 | NumPy • SciPy • quadprog • CasADi • Matplotlib

Contents

1 Introduction **2**

2 Installation **2**
2.1 Requirements . . . 2
2.2 Installing with pip (not available yet) . . . 2
2.3 Installing from source (Poetry) . . . 2
2.4 Installing using wheel file . . . 3

3 Package Structure **3**

4 Module: `utils.py` **3**
4.1 `calc_splines` . . . 3
4.2 `interp_splines` . . . 4
4.3 `calc_spline_lengths` . . . 4
4.4 `create_raceline` . . . 4
4.5 `calc_head_curv_an` . . . 4
4.6 `conv_filt` . . . 4
4.7 `import_veh_dyn_info` . . . 5
4.8 `H_f` . . . 5

5 Module: `solvers.py` **5**
5.1 `opt_min_curv` – Minimum-Curvature QP . . . 5
5.2 `ShortestPath` – Shortest Feasible Path . . . 6
5.3 Class `Opt_min_CurvTime` . . . 6
5.4 Class `ConstrainedCMAES_t` . . . 7
5.5 Class `ZORM` . . . 8
5.6 Class `Clothoid_raceline` . . . 8

6 Module: `kinematicProfs.py` **8**
6.1 `calc_vel_profile` . . . 9
6.2 `calc_ax_profile` . . . 9
6.3 `calc_t_profile` . . . 9
6.4 `curvature_profile` and `curvature_profile2` . . . 10
6.5 `cumulative_distances` . . . 10

7 Module: `opt_mintime.py` **10**

8 Usage Examples **10**

9 Usage Examples **10**
9.1 `General_test.py` – Geometric and Proxy Optimisation . . . 10
9.2 `General_test_optmintime.py` – Minimum Lap-Time Optimal Control . . . 12

10 GGV Diagram Format **14**

11 Track Map Format **14**

12 Dependencies and Licences **14**

1 Introduction

Racing-line optimisation is the problem of choosing, for every point around a closed circuit, a lateral position within the drivable corridor such that a chosen cost (lap time, path curvature, path length, ...) is minimised subject to track-boundary and vehicle-dynamics constraints.

OptiLine addresses this problem at multiple levels of fidelity:

1. **Geometric methods** – minimum-curvature (QP) and shortest-path (QP) formulations that operate purely on track geometry, producing a smooth spline raceline without requiring a vehicle model.
2. **Lap-time proxy methods** – combined curvature-and-time optimisers (`Opt_min_CurvTime`) that use the kinematic profile as a fast surrogate for lap time and drive the search with CMA-ES or ZORM.
3. **Optimal-control methods** – a full single-track / double-track minimum-time optimal-control problem solved by direct Gauss–Legendre collocation via CasADi and IPOPT (`opt_mintime`).

The package is structured as follows:

Module	Role
`solvers.py`	Core optimisation solvers and raceline classes
`utils.py`	Spline utilities, track preprocessing, QP matrix builders
`opt_mintime.py`	Minimum lap-time optimal-control problem (CasADi/IPOPT)
`kinematicProfs.py`	Velocity, acceleration, and lap-time profile computation

We should note that many parts of the codes are adopted from https://github.com/TUMFTM/global_racetrajectory_optimization.

2 Installation

2.1 Requirements

OptiLine requires Python 3.10–3.12 and the following libraries:

Library	Version	Purpose
numpy	$\geq 1.24, < 3$	Array operations
scipy	$\geq 1.10, < 2$	Interpolation, linear algebra
matplotlib	$\geq 3.7, < 4$	Plotting
quadprog	$\geq 0.1.13, < 0.2$	Quadratic programming
casadi	$\geq 3.7, < 4$	Automatic differentiation / IPOPT

2.2 Installing with pip (not available yet)

```
pip install optiline
```

2.3 Installing from source (Poetry)

```
git clone https://github.com/amirali78frz/OptiLine.git
cd OptiLine
poetry install
```

2.4 Installing using wheel file

```
pip install optiline-0.1.0-py3-none-any.whl
```

3 Package Structure

```
OptiLine/
|-- src/
|   |-- OptiLine/
|       |-- __init__.py
|       |-- solvers.py          # Optimisation algorithms
|       |-- utils.py            # Helper files
|       |-- opt_mintime.py      # Min-time optimal control
|       `-- kinematicProfs.py   # Velocity/acceleration/time profiles
|-- tests/
|   |-- General_test.py             # End-to-end geometric + proxy
    optimisation
|   `-- General_test_optmintime.py  # End-to-end min-time optimal
    control
|-- params/
|   `-- racecar.ini             # Vehicle dynamics parameters
|-- maps/                       # Track CSV files
|-- poetry.lock
`-- pyproject.toml
```

4 Module: `utils.py`

`utils.py` provides spline mathematics, track-preprocessing helpers, and QP matrix builders used by the solvers.

4.1 `calc_splines`

```
coeffs_x, coeffs_y, M, normvec_normalized = calc_splines(
    path,
    el_lengths=None,
    psi_s=None, psi_e=None,
    use_dist_scaling=True
)
```

Solves for curvature-continuous (C2) cubic spline coefficients through a sequence of track points. The spline passes through every point and matches heading and curvature at each knot. For a closed track, periodic boundary conditions are enforced.

Returns

Return	Description
`coeffs_x`	`(n,4)` cubic coefficients for the x-coordinate
`coeffs_y`	`(n,4)` cubic coefficients for the y-coordinate
`M`	Spline system matrix
`normvec_normalized`	`(n,2)` outward unit normals

4.2 interp_splines

```
path_interp, inds_interp, t_interp, dists_interp = interp_splines(
    coeffs_x, coeffs_y, spline_lengths,
    incl_last_point=False,
    stepsize_approx=None,
    stepsize_fixed=None,
)
```

Interpolates a 2-D spline at a uniform arc-length step size and returns sample points, segment indices, local parameters, and cumulative distances.

4.3 calc_spline_lengths

```
spline_lengths = calc_spline_lengths(
    coeffs_x, coeffs_y,
    quickndirty=False,
    no_interp_points=15
)
```

Numerically integrates the arc length of each cubic spline segment using Gaussian quadrature.

4.4 create_raceline

```
raceline, a_opt, coeffs_x_opt, coeffs_y_opt, spline_inds_opt,
t_vals_opt, s_points_opt, spline_lengths_opt, el_lengths_opt =
    create_raceline(
    refline, normvec_normalized, alpha,
    stepsize_interp
)
```

Applies lateral offsets α_i along normals to the reference line and recomputes fresh spline coefficients for the resulting raceline.

4.5 calc_head_curv_an

```
psi, kappa, dkappa = calc_head_curv_an(
    coeffs_x, coeffs_y,
    ind_spls, t_spls,
    calc_curv=True, calc_dcurv=False
)
```

Analytically evaluates heading ψ, curvature κ, and the curvature derivative $\dot{\kappa}$ from cubic spline coefficients at the given local parameters t.

4.6 conv_filt

```
signal_out = conv_filt(signal, filt_window, closed)
```

Applies a moving-average filter (box kernel) to a 1-D signal. `closed` controls whether the signal is treated as periodic.

4.7 import_veh_dyn_info

```
ggv, ax_max_machines = import_veh_dyn_info(
    ggv_import_path, ax_max_machines_import_path
)
```

Loads the GGV (grip–grip–velocity) diagram and the maximum traction force vs. velocity table from CSV files. The GGV is an $(n_v \times 3)$ array with columns $[v,\ a_{x,\max},\ a_{y,\max}]$.

4.8 H_f

```
H, f, G, h = H_f(A, kappa_bound)
```

Assembles the QP matrices $(H, \mathbf{f}, G, \mathbf{h})$ for the minimum-curvature problem. H is derived from the spline-curvature Hessian and G, $\mathbf{h}$ encode the track-boundary constraints on α.

5 Module: solvers.py

solvers.py contains the high-level optimisation classes and the lower-level QP-based geometric solvers.

5.1 opt_min_curv – Minimum-Curvature QP

```
alpha_mincurv, curv_error_max = opt_min_curv(
    reftrack, normvectors, A,
    kappa_bound, w_veh)
```

Solves a quadratic programm (QP) that minimises the integral of squared curvature of the resulting spline raceline. The optimisation variable is the vector $\boldsymbol{\alpha} \in \mathbb{R}^n$ of lateral shifts applied to each reference-line point along its outward normal.

Mathematical formulation

Given reference points $\mathbf{r}_i$ and unit normals $\hat{\mathbf{n}}_i$, the optimised track point is

$$\mathbf{p}_i = \mathbf{r}_i + \alpha_i \hat{\mathbf{n}}_i.$$

The QP minimises

$$\min_{\boldsymbol{\alpha}} \ \tfrac{1}{2}\boldsymbol{\alpha}^\top H \boldsymbol{\alpha} + \mathbf{f}^\top \boldsymbol{\alpha} \quad \text{subject to} \quad G\boldsymbol{\alpha} \le \mathbf{h},$$

where H and $\mathbf{f}$ encode spline curvature and the inequality constraints enforce track-boundary limits. The QP is solved with *quadprog*. More theoretical explanation can be found in [5].

Parameters

Parameter	Description
refline	(n,2) array – (x, y) coordinates of the reference line
normvec_normalized	(n,2) array – unit outward normal vectors
A	Spline coefficient matrix from **calc_splines**
kappa_bound	Maximum allowed curvature [m^{-1}]
w_veh	Vehicle width [m] used for safety margin
print_debug	Print solver status if True
plot_debug	Plot intermediate geometry if True

Returns

Return value	Description
alpha_mincurv	(n,) optimal lateral-shift vector
curv_error_max	Maximum curvature error vs. purely geometric bound

5.2 ShortestPath – Shortest Feasible Path

```
raceline,alpha_shpath,s_splines,vx_profile,ax_profile,kappa_opt,
    t_profile = ShortestPath(
     reftrack, w_veh, stepsize, plot, ggv_import_path,
    ax_max_machines_import_path
)
```

Solves a QP for the geometrically shortest path through the track corridor. The objective is the sum of squared Euclidean segment lengths. Theoretical explanation can be found in [1].

Parameters

Parameter	Description
refline	(n,2) reference line
normvec_normalized	(n,2) unit normals
w_tr_right	(n,) right half-widths [m]
w_tr_left	(n,) left half-widths [m]
w_veh	Vehicle width [m]

Returns

Return value	Description
alpha_shortest	(n,) lateral-shift vector

5.3 Class Opt_min_CurvTime

Opt_min_CurvTime combines spline geometry with a kinematic velocity profile to create a fast surrogate for lap time. A meta-optimiser (CMA-ES or ZORM) searches over the spline segment-length vector to minimise this surrogate.

Constructor

```
obj = Opt_min_CurvTime(
    reftrack,            # (n,4) array: x, y, w_right, w_left
    center,            # center line and track widths (same as
    reftrack for 1 pass runs)
    mu,                    # smoothing parameter of ZO optimiser
    kappab,         # max curvature [1/m]
    sfty,           half of the vehicle width (+ safty)
    si,               # interpolation step size of the race line
    vm,              # max velocity
    m_veh,              # vehicle mass
    drag_coeff,         # drag coefficient
    MC,          #Monte Carlo number of repetion for ZO solvers
    min_s,       #minimum curv length
    max_s,       #maximum curv length
```

```
    sigma,       #Initial covariance for CMA-es solver
    popsize,     #population size for CMA-es solver
    iterations_ZO,  #number of iterations for ZORM solver
    iterations_CMA, #number of iterations for CMA-es solver
    ggv_import_path=None,            # path to GGV diagram array
    ax_max_machines_import_path=None, # path to ax limits
    stepsize_opts=None, # dict of interpolation stepsizes
    reg_smooth_opts=None  # dict of smoothing options
)
```

Key methods

`f_t(s)` Objective function. Given a segment length vector `s`, reconstructs the spline raceline, computes the curvature, integrates a kinematic velocity profile, and returns the predicted lap time t_{lap}.

`CurveLenOpt(method='ZO', ...)` Runs the meta-optimisation. `method` may be `'ZO'` (ZORM) or `'CMAES'` (CMA-ES). Returns the optimised segment-length vector and the corresponding lap time.

`generate_raceline(s_opt)` Given an optimised segment-length vector, returns the full raceline geometry: reference points, spline coefficients, curvature, heading, arc lengths, and the α vector, obtained by solving the minimum curvature problem but with given segment lengths. Running this function with the Euclidean distance between the points as the segment length recovers the minimum curvature solution.

`generate_kinProfs(raceline_data)` Computes velocity, acceleration, and lap-time profiles from the raceline geometry using the forward-backward method. Running this function with the Euclidean distance between the points as the segment length recovers the minimum curvature solution.

`Comparison(...)` Prints a table comparing the lap times of the ZO, CMA-ES, minimum-curvature, and centre-line solutions, and produces a comparison plot.

5.4 Class `ConstrainedCMAES_t`

A self-contained implementation of CMA-ES (Covariance Matrix Adaptation Evolution Strategy) adapted for constrained problems via death-penalty constraint handling.

Constructor

```
cma = ConstrainedCMAES_t(
    f,                # callable: objective f(x) -> float
    mean,               # initial mean vector
    sigma0=0.3,       # initial step-size
    popsize=None,     # population size (default: 4 + floor(3*ln(n)))
    bounds1,          # Upper bounds
    bounds2,          # Lowe bounds
)
s_opt = cma.optimize(iterations)
```

Algorithm outline

Each generation the algorithm:

1. Samples λ candidate solutions from $\mathcal{N}(\mathbf{m}, \sigma^2 C)$.
2. Evaluates f.
3. Ranks candidates and updates the mean $\mathbf{m}$, step-size σ, and covariance C via cumulative step-size adaptation (CSA) and cumulative path-length control (CMA).

More theoretical explanation can be found in [4].

5.5 Class ZORM

Zero-Order Random Method (ZORM) is a gradient-free optimiser that estimates a smoothed gradient using random perturbations.

Constructor

```
opt = ZORM(
    func,           # callable: objective f(x) -> float
    x0,          # initial point
    num_iterations, #number of iterations
    mu=0.05,      # smoothing parameter
    h=0.001,     # learning rate
    grad_type='noth', # 'noth' for forward estimator and 'h' for difference estimator
    constraint_type='c',    # 'c' for having box constraints and 'notc' for opposite
)
x_opt = opt.optimize(lower_bound, upper_bound)
```

Algorithm outline

At each iteration, ZORM estimates the gradient of the μ-smoothed objective

$$f_\mu(\mathbf{x}) = \mathbb{E}_{\mathbf{u}\sim\mathcal{N}(0,I)}[f(\mathbf{x} + \mu\mathbf{u})]$$

by averaging finite-difference directional derivatives along a random direction, then takes a step in the estimated negative gradient direction. More theoretical explanation can be found in [6].

5.6 Class Clothoid_raceline

Fit a piecewise clothoid (Euler spiral) raceline where curvature varies linearly along each segment.

```
cl = Clothoid_raceline(kappa0, kappa1, length)
x_pts = cl.X(s_array)    # x-displacements at arc-length positions
y_pts = cl.Y(s_array)    # y-displacements at arc-length positions
```

The Fresnel-type integrals are evaluated numerically via *scipy.integrate*. More details can be found in [3]

6 Module: kinematicProfs.py

kinematicProfs.py computes physically consistent velocity, acceleration, and lap-time profiles along a given raceline curvature profile, using a GGV diagram or simple $a_{x,\max}/a_{y,\max}$ limits.

6.1 calc_vel_profile

```
vx_profile = calc_vel_profile(
    ggv, ax_max_machines,
    v_max, kappa, el_lengths, closed,
    drag_coeff=0.0, m_veh=1000.0,
    dyn_model_exp=1.0,
    mu=None,
    v_start=None, v_end=None,
)
```

Computes the maximum feasible velocity profile using a forward–backward integration algorithm:

1. **Lateral limit**: At each point, the lateral acceleration from curvature limits the speed:

$$v_i \leq \sqrt{a_{y,\max}(v_i)/|\kappa_i|}.$$

2. **Forward pass**: Accelerate from a start speed while respecting $a_{x,\max}$ and the GGV diamond.
3. **Backward pass**: Decelerate from an end speed to enforce braking constraints.
4. **Final profile**: Element-wise minimum of the lateral limit, forward, and backward profiles.

Key parameters

Parameter	Description
`ggv`	GGV array `(n,3)`: $[v, a_x, a_y]$
`ax_max_machines`	Motor traction table `(m,2)`: $[v, a_{x,\text{drive}}]$
`v_max`	Hard speed cap [m/s]
`kappa`	Curvature array [m^{-1}]
`el_lengths`	Segment lengths [m]
`closed`	`True` for a closed lap; iterate to convergence
`drag_coeff`	Aerodynamic drag coefficient [kg/m]
`m_veh`	Vehicle mass [kg]

6.2 calc_ax_profile

```
ax_profile = calc_ax_profile(vx_profile, el_lengths, eq_length_output
    =False)
```

Numerically differentiates the velocity profile to give the longitudinal acceleration $a_x = \Delta v^2/(2\,\Delta s)$ at each point.

6.3 calc_t_profile

```
t_profile = calc_t_profile(vx_profile, el_lengths, t_start=0.0,
    ax_profile = None)
```

Integrates the velocity profile to give the elapsed time at each sample point. Returns a vector of length $n+1$ (including the finish-line time).

6.4 curvature_profile and curvature_profile2

```
kappa = curvature_profile(points)        # numerical cross-product method
kappa = curvature_profile2(reftrack)   # spline-derivative method
```

Two alternative functions for computing the curvature of a discrete point sequence. **curvature_profile** uses *scipy.interpolate.splprep* and **curvature_profile** does not.

6.5 cumulative_distances

```
s = cumulative_distances(path)
```

Returns the cumulative Euclidean arc-length from the first point, starting at zero.

7 Module: opt_mintime.py

opt_mintime solves a full minimum-lap-time optimal-control problem (OCP) formulated in the curvilinear frame of the track. It uses CasADi to construct the nonlinear programme (NLP) symbolically and IPOPT as the NLP solver. More theoretical explanation can be found in [2].

Function signature

```
result = opt_mintime(
    reftrack,                # (n,4): x, y, w_right, w_left
    coeffs_x,                # spline x-coefficients (n,4)
    coeffs_y,                # spline y-coefficients (n,4)
    normvectors,      # unit normals (n,2)
    pars,                    # dict of vehicle and solver parameters
    print_debug=False,
    plot_=False,
)
```

8 Usage Examples

9 Usage Examples

The following two scripts, located in the `tests/` directory, provide complete end-to-end demonstrations of the package. Each script is organised into numbered stages; the titles below correspond directly to the printed stage headers.

9.1 General_test.py – Geometric and Proxy Optimisation

Run from the `tests/` directory:

```
cd tests && python General_test.py
```

Stage 1 – Track import and preprocessing

Loads the Melbourne centreline CSV, sub-samples it to reduce computation time, and imports the GGV and `ax_max_machines` tables.

```
MAP_PATH = "maps/Melbourne/Melbourne_centerline.csv"
csv_data = np.loadtxt(MAP_PATH, comments='#', delimiter=',')
reftrack_full = csv_data[:, 0:4]          # [x, y, w_right, w_left]
reftrack = subsample_track(reftrack_full, step=4)
ggv, ax_max_machines = import_veh_dyn_info(GGV_PATH, AX_MAX_PATH)
```

Stage 2 – Spline computations

Fits C2-continuous cubic splines through the reference points, recovers segment lengths, interpolates at a uniform 2 m step, and computes analytical and numerical heading and curvature profiles.

```
coeffs_x, coeffs_y, M, normvec_norm = calc_splines(
    path=np.vstack((reftrack[:, :2], reftrack[0, :2])),
    el_lengths=ds_0, use_dist_scaling=True)
spline_lengths = calc_spline_lengths(coeffs_x, coeffs_y)
path_interp, spline_inds, t_vals, s_pts = interp_splines(
    coeffs_x, coeffs_y, spline_lengths, stepsize_approx=2.0)
psi_an, kappa_an = calc_head_curv_an(coeffs_x, coeffs_y,
                                     spline_inds, t_vals)
```

Stage 3 – Minimum-curvature raceline

Solves the curvature-minimisation QP to find the optimal lateral shifts $\boldsymbol{\alpha}$, then reconstructs the raceline and computes its velocity and lap-time profiles.

```
alpha_mincurv, curv_error = solvers.opt_min_curv(
    reftrack=reftrack, normvectors=normvec_norm, A=M,
    kappa_bound=0.5, w_veh=1.0)
raceline_mc, *_ = create_raceline(
    reftrack[:, :2], normvec_norm, alpha_mincurv, stepsize_interp
    =2.0)
vx_mc  = calc_vel_profile(ggv, ax_max_machines, v_max=22.88,
                          kappa=kappa_mc, el_lengths=el_mc, closed=
    True,
                          drag_coeff=0.75, m_veh=1000)
t_mc   = calc_t_profile(vx_profile=vx_mc, ax_profile=ax_mc,
                        el_lengths=el_mc)
```

Stage 4 – Shortest-path raceline

Solves the shortest-path QP and returns the full raceline geometry together with pre-computed kinematic profiles.

```
raceline_sp, alpha_sp, s_sp, vx_sp, ax_sp, kappa_sp, t_sp = \
    solvers.ShortestPath(
        reftrack=reftrack, w_veh=1.0, stepsize=2.0, plot=False,
        ggv_import_path=GGV_PATH,
        ax_max_machines_import_path=AX_MAX_PATH)
```

Stage 5 – Kinematic profiles

Demonstrates the **kinematicProfs** module in isolation using the minimum-curvature raceline from Stage 3: plots velocity, longitudinal acceleration, and curvature against arc length.

Stage 6 – Full optimisation comparison (ZO vs CMA-ES)

Instantiates **Opt_min_CurvTime**, evaluates the objective at the initial segment lengths, then runs **generate_raceline** and **generate_kinProfs** with the ZORM solver before calling **Comparison** to benchmark ZO, CMA-ES, minimum-curvature, and centre-line solutions side by side.

```
optct = solvers.Opt_min_CurvTime(
    reftrack=reftrack, center=reftrack, mu=0.01, h=0.001,
    kapb=0.5, sfty=1.0, si=2, vm=22.88, m_veh=1000,
    drag_coeff=0.75, MC=1, min_s=0.5, max_s=2.0,
    sigma=0.005, iterations_ZO=100, iterations_CMA=10,
    popsize=16, ggv_import_path=GGV_PATH,
    ax_max_machines_import_path=AX_MAX_PATH, fw=3)

raceline_zo, ds_zo = optct.generate_raceline(solver='ZO')
s_zo, vx_zo, ax_zo, kappa_zo, t_zo, rl_zo = \
    optct.generate_kinProfs(ds=ds_zo)

s_splines, vx_opt, ax_opt, kappa_opt, t_opt, rl_opt = \
    optct.Comparison(plot='Y', output='ZO')
```

Stage 7 – Re-optimisation on a refined reference track

Uses the ZO-optimised raceline as the new reference, sub-samples it at a finer resolution, and runs a second ZORM pass to check for additional lap-time savings.

```
reftrack_new = subsample_track(reftrack_new_full, step=2)
optct_reopt  = solvers.Opt_min_CurvTime(reftrack=reftrack_new,
                    center=reftrack_new, ...)  # same hyper-parameters
ds_reopt = optct_reopt.CurveLenOpt(solver='ZO')
s_ro, vx_ro, ax_ro, kappa_ro, t_ro, rl_ro = \
    optct_reopt.generate_kinProfs(ds=ds_reopt)
print(f"Improvement: {t_opt[-1] - t_ro[-1]:+.2f} s")
```

9.2 General_test_optmintime.py – Minimum Lap-Time Optimal Control

Run from the `tests/` directory:

```
cd tests && python General_test_optmintime.py
```

Stage 1 – Track import and sub-sampling

Loads and sub-samples the Melbourne centreline identically to `General_test.py` (Stage 1), reducing the number of collocation intervals for faster NLP solving.

Stage 2 – Configuration loading

Parses `params/racecar.ini` with `configparser` to populate all vehicle and solver parameter dictionaries, and loads the GGV and `ax_max_machines` tables. Safety acceleration limits (`ax_pos_safe`, `ax_neg_safe`, `ay_safe`) are derived automatically from the GGV table when left as `None` in the INI file.

```
parser = configparser.ConfigParser()
parser.read("params/racecar.ini")
pars["vehicle_params_mintime"] = json.loads(
    parser.get('OPTIMIZATION_OPTIONS', 'vehicle_params_mintime'))
pars["tire_params_mintime"] = json.loads(
    parser.get('OPTIMIZATION_OPTIONS', 'tire_params_mintime'))
if pars["optim_opts"]["ax_pos_safe"] is None:
    pars["optim_opts"]["ax_pos_safe"] = np.amin(ggv[:, 1])
```

Stage 3 – Track preprocessing

Calls **`utils.prep_track`** to smooth the centreline with B-spline approximation, re-sample at the collocation stepsize, and return the interpolated track, unit normals, and spline coefficients.

```
reftrack_interp, normvec_normalized_interp, a_interp, \
    coeffs_x_interp, coeffs_y_interp = utils.prep_track(
        reftrack_imp=reftrack,
        reg_smooth_opts=pars["reg_smooth_opts"],
        stepsize_opts=pars["stepsize_opts"],
        debug=True)
```

Stage 4 – Minimum lap-time optimisation (pass 1)

Solves the full OCP via Gauss–Legendre collocation and IPOPT. Returns the optimal lateral shift $\boldsymbol{\alpha}$, velocity profile, raceline geometry, and arc-length vector.

```
alpha_opt, v_opt, raceline_opt, reftrack_interp_out, \
    a_interp_tmp, normvec_out, s_opt, laptime = \
    opt_mintime.opt_mintime(
        reftrack=reftrack_interp,
        coeffs_x=coeffs_x_interp, coeffs_y=coeffs_y_interp,
        normvectors=normvec_normalized_interp,
        pars=pars, print_debug=True, plot_=True)
```

Stage 5 – Re-optimisation pass (widened corridor)

Widens the optimisation corridor by $\Delta w = w_{\text{tr,reopt}} - w_{\text{veh,reopt}} + w_{\text{add,spl}}$ and re-solves the NLP using the pass-1 reference track and normals as the new starting geometry.

```
w_veh_tmp = (pars["optim_opts"]["width_opt"]
             + pars["optim_opts"]["w_tr_reopt"]
             - pars["optim_opts"]["w_veh_reopt"]
             + pars["optim_opts"]["w_add_spl_regr"])
pars_reopt = copy.deepcopy(pars)
pars_reopt["optim_opts"]["width_opt"] = w_veh_tmp

alpha_reopt, v_reopt, raceline_reopt, *_, s_reopt, laptime_reopt = \
    opt_mintime.opt_mintime(
        reftrack=reftrack_interp_out,
        normvectors=normvec_out,
        pars=pars_reopt, ...)
print(f"Pass 1: {laptime:.2f} s   Pass 2: {laptime_reopt:.2f} s")
```

Stage 6 – Results inspection and plots

Produces three figures: (a) pass-1 and re-optimised racelines overlaid on the centreline, (b) velocity profiles vs. arc length, (c) lateral shift α profiles vs. arc length. A summary table is printed to the console.

10 GGV Diagram Format

The GGV (velocity–lateral-g–longitudinal-g) file must be a comma-separated file with **no header row**:

Column 1 v [m/s]	Column 2 $a_{x,\max}$ [m/s^2]	Column 3 $a_{y,\max}$ [m/s^2]
0.0	10.0	12.0
10.0	10.0	11.5
20.0	9.5	10.0
...	...	...

The `ax_max_machines` file has two columns: $[v,\ a_{x,\text{drive}}]$.

11 Track Map Format

Track map CSV files must have **four columns** and **no header**:

Column 1 x [m]	Column 2 y [m]	Column 3 w_{right} [m]	Column 4 w_{left} [m]
0.00	0.00	5.0	5.0
5.23	0.12	5.0	5.0
...	...	...	...

The track is assumed **closed**: the last point connects back to the first. Do not repeat the first point at the end.

12 Dependencies and Licences

Library	**Licence**	**URL**
NumPy	BSD-3-Clause	https://numpy.org
SciPy	BSD-3-Clause	https://scipy.org
Matplotlib	Matplotlib	https://matplotlib.org
quadprog	GPL-2.0	https://github.com/quadprog/quadprog
CasADi	LGPL-3.0	https://web.casadi.org
IPOPT	EPL-2.0	https://coin-or.github.io/Ipopt

12 References

[1] Francesco Braghin, Federico Cheli, Stefano Melzi, and Edoardo Sabbioni. Race driver model. *Computers & Structures*, 86(13-14):1503–1516, 2008.

[2] Fabian Christ, Alexander Wischnewski, Alexander Heilmeier, and Boris Lohmann. Time-optimal trajectory planning for a race car considering variable tyre-road friction coefficients. *Vehicle system dynamics*, 59(4):588–612, 2021.

[3] Marco Frego, Enrico Bertolazzi, Francesco Biral, Daniele Fontanelli, and Luigi Palopoli. Semi-analytical minimum time solutions with velocity constraints for trajectory following of vehicles. *Automatica*, 86:18–28, 2017.

[4] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

[5] Alexander Heilmeier, Alexander Wischnewski, Leonhard Hermansdorfer, Johannes Betz, Markus Lienkamp, and Boris Lohmann. Minimum curvature trajectory planning and control for an autonomous race car. *Vehicle System Dynamics*, 2020.

[6] Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. 17(2):527–566.